

Tide retrospective:

Cutting Multi/Covariance.lean compile time by removing nlinarith

Claude (Opus 4.8) with Adam Newgas

2026-06-26

Abstract

This is a maintenance tide rather than a formalisation tide: the seabed gains no new theorem, but its slowest-compiling file gets faster. The multivariate Laplace-covariance file—the Stage 3 payoff that proves $\text{Cov}_t[\varphi, \psi] = \frac{1}{t} \langle \nabla \varphi, \Sigma \nabla \psi \rangle + O(t^{-2})$ through a stack of asymptotic integral bounds—was taking 36s wall / 107s CPU to elaborate. Profiling showed that *five* `nlinarith` calls accounted for roughly 22s of that CPU; all five proved goals that are either linear given a hint already in hand or are a single named Mathlib lemma. Replacing them (`nlinarith` → `linarith`, `le_mul_of_one_le_right`, `le_self_pow`) dropped the file to 28.4s wall / 82s CPU—a 21% wall and 23% CPU reduction—with a five-line diff and no change to any proven statement. The headline lesson is mundane and reusable: in a heartbeat-heavy Lean file, `nlinarith` on a goal that is secretly linear is the first thing to grep for.

1 Setting

The broader programme is the formalisation of the SLT Susceptibility Primer’s “boring” Laplace calculations in the `laplace` seabed. Its Stage 3 deliverable is the multivariate covariance asymptotic $\text{Cov}_t[\varphi, \psi] = \frac{1}{t} \langle \nabla \varphi, \Sigma \nabla \psi \rangle + O(t^{-2})$, and the file that carries the heavy lifting for it is `Laplace/Multi/Covariance.lean`: 3904 lines, a single import, one 3600-line section of asymptotic integral bounds, with `maxHeartbeats` cranked to 800000. It is the slowest file in the repo to elaborate, so it is the natural target when the direction is simply “make this compile faster.”

This tide deviates from the standard template in one structural way, noted here once: there is no “thing formalised.” The deliberation in Step 2 was not about choosing a minimal lemma but about confirming that a proposed performance refactor is sound and is the highest-impact available move. Accordingly the “thing formalised” section below is repurposed as “the change,” and “proof strategy” as “why the calls were slow.”

The candidate was chosen by measurement, not taste. A single profiled build (`lean -Dprofiler=true`) makes the cost distribution unambiguous: the five `nlinarith` invocations dominate everything else by an order of magnitude, with the next tier (the 54 `positivity` calls, 20 `field_simp` calls) spread thinly at 0.1–1.4s each. When one tactic family is 60% of the CPU budget, that is the minimal good target.

2 The change

Five `nlinarith` calls were replaced, in three shapes. None alters a statement; the file still builds with only the pre-existing `push_neg` deprecation warnings.

Shape A (two calls). Inside the rescaled-weight remainder bounds,¹ the goal at the call is

$$\frac{c}{2} \|u\|^2 + \frac{cR_\varphi^2}{2} t \leq c \|u\|^2,$$

with a hypothesis `h_half_le`: $\frac{c}{2}(R_\varphi^2 t) \leq \frac{c}{2}\|u\|^2$ already in hand. Treating $c\|u\|^2$ and $cR_\varphi^2 t$ as atoms, the goal *is* that hypothesis rearranged—purely linear. `nlinarith` was forming degree-2 products across the large surrounding context for nothing.

`nlinarith [h_half_le] → linarith [h_half_le].`

Shape B (two calls). Inside a polynomial-growth helper, branch $1 < \|u\|$, the goal after `rw [pow_succ]` is $\|u\|^k \leq \|u\|^k \cdot \|u\|$. This is exactly the ordered-semiring lemma `le_mul_of_one_le_right`:

`nlinarith [pow_nonneg (norm_nonneg u) k]
→ exact le_mul_of_one_le_right (pow_nonneg ...) hu.le.`

Shape C (one call). Inside the largest single proof,² a nested goal $t \leq t^2$ proved by `nlinarith`, with `ht1 : 1 ≤ t` in scope, is exactly the named lemma `le_self_pow0`. The fix replaces the tactic proof with the term `le_self_pow0 ht1` (by `norm_num`) supplied directly to `Real.sqrt_le_sqrt`.

Metric	Before	After	Δ
Wall clock	36.1 s	28.4 s	−7.7 s (−21%)
CPU (user)	107 s	82 s	−25 s (−23%)
<code>nlinarith</code> calls	5	0	−5

All measurements: lake env lean Laplace/Multi/Covariance.lean in the tide worktree, warm Mathlib cache, same machine; figures stable across repeated runs.

3 Why the calls were slow

`nlinarith` extends `linarith` with a Positivstellensatz pre-processing step: it forms pairwise products of the hypotheses (and their squares) to give the linear arithmetic backend access to degree-2 facts. That product set grows quadratically in the number of in-context hypotheses, and these goals live deep inside 300–600-line proofs with dozens of `haves` in scope. So each call paid for a large product search to prove something that needed at most one product—or none. The four “Shape A/B” calls cost 2.7–9.3 s each; “Shape C” cost 3.9 s.

The interesting wrinkle is *where* the savings show up. CPU dropped by 25 s but wall by only 7.7 s, because `lean` elaborates declarations across several cores (~ 3 here, from the 107/36 CPU-to-wall ratio). The four Shape-A/B calls sit in *distinct* lemmas that elaborate concurrently with other heavy work, so removing them cut total work far more than latency. Shape C is the exception: it lives inside the single largest proof, which is itself the wall-clock critical path, so fixing that one call alone bought ~ 4.5 s of the 7.7 s wall reduction. The general lesson—profile for *which declaration is on the critical path*, not just which tactic is expensive in aggregate—is recorded below.

¹In `abs_integral_remainder_mul_rescaled_weight_le` and `abs_integral_dot_mul_remainder_mul_rescaled_weight_le`.

²The 598-line `abs_integral_remainder_mul_remainder_mul_rescaled_weight_le`.

4 Roadblocks and resolutions

A false “credentials missing” preflight. The GPT-5.5 Pro consult nearly did not happen. The preflight test `-f .../credentials.json` reported the file missing, and I almost proceeded down the “no-GPT” path. Two compounding causes: first, an earlier `cd` into the Lean repo had persisted across `Bash` calls, so the relative-path test ran from the wrong directory; second—surfaced only when the consult itself failed—the `credentials.json` that *does* exist had a trailing comma after the `api_key`, making it invalid JSON, so the script’s legacy fallback silently bailed. Adam caught the first error (“the legacy credentials.json is there”); the second was a one-character fix once the parse error was read. Lesson recorded: preflight existence checks should use absolute paths, and a malformed-but-present credential file is a distinct failure mode from a missing one.

Mis-estimating the fifth call. The first deliberation draft labelled the Shape-C call as the “cheap, ~ 0.5 s one, left as-is” and proposed touching only the other four. Re-profiling *after* the first four edits showed a 3.9 s `nlinarith` still present—it had been mis-attributed. The fix is procedural: re-profile after each batch of edits rather than trusting the initial attribution of aggregate costs to specific lines. Catching it mattered: Shape C was the one on the critical path.

One refinement from GPT. For this tide the GPT-5.5 Pro consult was an adversarial soundness review (“do these replacements close exactly these goals, and is `nlinarith` the right target?”) rather than a strategy generator. It confirmed all three shapes sound—in particular that `linarith` normalises the $cR_\phi^2 t$ monomial without a `ring` bridge, and that `le_mul_of_one_le_right` is the correct current name—and agreed `nlinarith` was the right target. Its one substantive addition was a guard worth recording:

GPT-5.5 Pro: `linarith [h_half_le]` still scans and uses the whole local context; `only` is the important compile-time guard. For deep proofs with huge contexts, this often changes runtime by orders of magnitude.

So the two Shape-A calls became `linarith only [h_half_le]`. In this instance the measured time did not change (the goal’s context after the `rw` is already small), but the principle—prefer `linarith only` and `nlinarith only` in large-context proofs—is the durable lesson, and the `only` form guards against context growth in future edits. GPT also flagged, and I deferred, the secondary observations that `maxHeartbeats` does not affect speed and that splitting the file aids incremental rebuilds but not a single slow tactic.

5 What was Mathlib, what was new

Entirely Mathlib lookup; nothing new was synthesised. The replacements lean on `linarith` (with the existing hint), `le_mul_of_one_le_right`, `le_self_pow0`, `pow_nonneg`, and `norm_nonneg`—all standard ordered-algebra API. The only “new” artefact is negative: five fewer tactic invocations. Candidate replacements were pre-checked in a scratch file before editing, so the single full rebuild was confirmation rather than search.

6 Lessons

- **For the seabed CLAUDE.md.** `nlinarith` on a goal that is linear-given-a-hint, or that is a single named inequality lemma, is a compile-time smell: it pays for a quadratic product

search to prove something cheap. Prefer `linarith only [hint]` or the named lemma (`le_mul_of_one_le_right`, `le_self_pow0`, etc.). In this file that one substitution was worth $\sim 23\%$ of CPU. And when an arithmetic tactic *is* needed in a large-context proof, reach for the only form (`linarith only [...]`, `nlinarith only [...]`): it proves from the listed facts plus the negated goal instead of scanning the whole local context, which for deep proofs can change runtime by orders of magnitude.

- **For the lean-formalisation skill.** When optimising compile time, profile for the critical-path *declaration*, not just the aggregate-expensive tactic. CPU savings and wall savings diverge under `lean`'s parallel elaboration; only fixes on the longest single proof move the wall clock. And re-profile after every edit batch—initial attribution of aggregate costs to individual lines is unreliable.
- **For the tide skill.** Preflight credential checks should use absolute paths and should distinguish “file absent” from “file present but unparseable.” A persisted shell `cd` silently invalidated a relative-path check here.

7 Follow-ups

- **The next compile-time target** in this file is the 54 `positivity` calls (`evalMul`), individually 0.1–1.4s. Worth a profiled pass to see whether any sit on the critical-path proof and whether explicit `positivity` extensions or term-mode `mul_pos` chains are cheaper.
- **Revisit `maxHeartbeats 800000`** once the heavy tactics are trimmed: check whether the budget can drop back toward the default (a lower cap does not speed compilation, but a tighter one documents the real requirement and guards against future regressions).
- **Sweep sibling files** (`Multi/RescaledIntegrals.lean`, `Multi/CovarianceSharp.lean`) for the same `nlinarith-on-linear` pattern.

Acknowledgements

GPT-5.5 Pro provided the adversarial soundness review of the replacements (full transcript in the tide log). Adam Newgas set the direction and caught the false-negative credentials preflight.